

TheOmniStack – Architektur & Hosting

Dokumentation (v2)

Stand: Mai 2026

Projekt: TheOmniStack (Multi-Tenant SaaS für E-Commerce)

1. Einleitung

TheOmniStack ist eine moderne, mandantenfähige (Multi-Tenant) Software-as-a-Service-Plattform. Sie ermöglicht E-Commerce-Händlern die Zentralisierung ihres Bestellmanagements, die Automatisierung der Rechnungsstellung sowie eine nahtlose, übergreifende Label-Erstellung bei verschiedenen Versanddienstleistern.

2. Technologie-Stack (Core)

Die Plattform basiert auf modernsten Web-Technologien für maximale Performance, Typ-Sicherheit und Skalierbarkeit:

- **Frontend & API-Layer:** Next.js (App Router) mit React
 - **Sprache:** TypeScript (Fullstack) für fehlerresistenten Code
 - **Styling:** Tailwind CSS für schnelle, responsive und moderne Benutzeroberflächen
 - **Datenbank:** PostgreSQL
 - **ORM (Object-Relational Mapping):** Drizzle ORM für typsichere Datenbankabfragen
 - **Background Jobs & Queues:** BullMQ mit Redis (zur asynchronen Abarbeitung von Bestellimporten und Synchronisationen)
-

3. Infrastruktur & Hosting-Umgebungen

Um Ausfallsicherheit, Datenschutz und einen reibungslosen Entwicklungsprozess zu garantieren, wird TheOmniStack in zwei strikt voneinander getrennten Umgebungen betrieben.

3.1 Produktiv-Umgebung (Production)

Die Live-Umgebung für alle aktiven Händler und Mandanten. Hier liegt der Fokus auf Hochverfügbarkeit (High Availability) und automatischer Skalierung.

- **Frontend & Serverless API (Hosting):** Vercel. Vercel übernimmt das globale CDN-Caching, das SSL-Management und die automatische Skalierung der Next.js Server Actions.
- **Hauptdatenbank:** Managed PostgreSQL-Datenbank. Regelmäßige automatisierte Backups und Point-in-Time-Recovery schützen vor Datenverlust.
- **Redis-Cluster:** Ein Managed Redis-Service speichert die BullMQ-Warteschlangen für die Hintergrundprozesse zwischen.
- **Worker-Prozesse (Background Sync):** Dedizierte Node.js-Dienste (Worker), die unabhängig vom Web-Frontend laufen. Sie arbeiten kontinuierlich die Redis-Queues ab, um im Minutentakt Bestellungen von Marktplätzen (wie Otto oder Amazon) herunterzuladen, ohne das Web-Interface der Nutzer auszubremsen.
- **Externe APIs:** Sämtliche Adapter (DHL, Hermes, Otto, Mirakl) kommunizieren ausschließlich mit

den Live-/Produktions-Endpunkten der jeweiligen Drittanbieter.

3.2 Test- & Entwicklungs-Umgebung (Staging / Development)

Diese Umgebung dient der agilen Weiterentwicklung und Qualitätssicherung (QA), bevor neue Funktionen für Kunden freigeschaltet werden.

- **Lokale Entwicklung (Docker):** Die Entwicklung findet isoliert in Docker-Containern (z. B. dem easybill_dev Container) statt. Dadurch wird gewährleistet, dass jeder Entwickler auf exakt demselben System-Setup arbeitet, das später in Produktion geht.
 - **Preview-Deployments:** Durch die CI/CD-Pipeline in Vercel wird bei jedem Code-Push (z. B. einem Pull Request) eine isolierte Staging-URL generiert. So können neue Features im Team oder mit Stakeholdern getestet werden, bevor sie in den main-Branch wandern.
 - **Isolierte Datenbanken:** Die Staging-Umgebung ist mit einer komplett eigenständigen PostgreSQL-Testdatenbank verbunden. Echte Kundendaten bleiben unangetastet.
 - **Sandbox-APIs:** Alle Marktplatz- und Versand-Integrationen (Amazon, Otto, Mirakl, DHL, Hermes) sind im System-Backend auf den Modus environment: 'sandbox' konfigurierbar. In dieser Einstellung kommunizieren die internen Adapter ausschließlich mit den Test- und Sandbox-Servern der Drittanbieter (z. B. <https://sandbox.api.otto.market>), sodass keine echten Bestellungen verfälscht oder echte Versandkosten produziert werden.
-

4. Systemarchitektur: Integrationen (Adapter Pattern)

Das Herzstück von TheOmniStack ist die modulare Anbindung von Verkaufskanälen und Logistikern. Das System nutzt das **Adapter Pattern**, wodurch neue Plattformen standardisiert angebunden werden können.

4.1 Marktplätze & Shopsysteme (Inbound)

Der Import von Bestellungen und der Rückversand von Versandbestätigungen ist für folgende Systeme standardisiert:

- **Amazon EU:** Anbindung via Amazon Selling Partner API (SP-API) mit OAuth2 und Token-Rotation.
- **Otto Partner Connect:** Strenge REST-API (v4) mit Basic Auth für OAuth2-Token-Austausch.
- **Mirakl (z.B. Decathlon & Custom):** Dynamische Architektur, die es erlaubt, beliebig viele Mirakl-basierte Marktplätze (wie Limango, Worten etc.) über dieselbe Logik (mit variablen Endpunkten) anzubinden.
- **Shopify & About You:** Direkte API-Kommunikation via Admin-API-Tokens.

4.2 Versanddienstleister (Outbound)

Die Logistik-Adapter übernehmen die automatisierte Label-Erstellung.

- **DHL (Post & Parcel Germany v2):** API-Gateway-Anbindung mittels Basic Auth und App-Secret. Unterstützt nationale/internationale Zonen, Zusatzservices sowie eine komplexe Logik für Retouren-Labels (Online vs. Beilage).
 - **Hermes:** Sichere Anbindung über das Hermes HSI-Portal via OAuth.
 - **Routing-Logik:** Die Plattform entscheidet intelligent pro Marktplatz (z.B. bei Otto), ob dem Paket ein Retouren-Label beigelegt werden muss oder ob der Marktplatz die Retoure eigenständig abwickelt.
-

5. Sicherheit & Datenschutz (Compliance)

- **Multi-Tenancy (Mandantenfähigkeit):** Die Architektur erzwingt auf Datenbank-Ebene (durch Drizzle-ORM-Filter und die Auth-Session companyld) eine absolute Trennung aller Mandanten. Ein Nutzer hat niemals Zugriff auf Rechnungen oder API-Schlüssel eines anderen Händlers.
 - **Verschlüsselung:** Sensible API-Keys und Secrets der Drittanbieter (DHL, Amazon etc.) werden verschlüsselt gespeichert und übertragen.
 - **Finanz-Compliance (GoBD):** Um den strengen Anforderungen der Buchhaltung gerecht zu werden, implementiert TheOmniStack lückenlose Rechnungsjournale und detaillierte CSV/DATEV-Exporte. Die USt-IdNr.-Validierung ist nativ integriert.
-

6. Backup & Disaster Recovery

Um Datenverlust vorzubeugen und eine hohe Ausfallsicherheit zu gewährleisten, ist eine automatisierte Backup-Strategie implementiert:

- **Tägliches automatisiertes Backup (GitHub Actions):** Ein Cron-Job wird jede Nacht (um 03:00 Uhr UTC) über GitHub Actions ausgeführt.
- **Datenbank-Backup (PostgreSQL):** Die Hauptdatenbank (Neon) wird mittels pg_dump exportiert. Das resultierende .sql.gz-Archiv wird sicher in einen separaten Backup-Bucket auf AWS S3 geladen. Über eine **AWS S3 Lifecycle Rule** werden Datenbank-Backups automatisch nach 30 Tagen gelöscht, um Speicherplatz und Kosten zu optimieren.
- **Dokumenten-Backup (S3):** Alle im Produktions-S3-Bucket generierten GoBD-konformen Rechnungen und Dokumente werden via aws s3 sync in den dedizierten Backup-Bucket gespiegelt. (Diese Dokumente sind von der Lifecycle Rule ausgenommen und bleiben permanent erhalten).
- **Benachrichtigungen:** Nach jedem erfolgreichen oder fehlgeschlagenen Backup-Lauf wird automatisch eine Status-E-Mail (via Resend) an den Administrator versendet, um die Integrität der Backups zu überwachen.
- *(Redis/Queue-Daten werden bewusst nicht persistent gesichert, da diese nur für temporäre Background-Jobs genutzt werden).*